

MATERIAL DIDÁCTICO — SESIÓN 1

Control de versiones con Git, GitHub y VS Code (Parte 1): repositorio, staging y commits

DESARROLLO — 2.1 Marco teórico (≈ 60 minutos)

¿Qué es un sistema de control de versiones (VCS)?

Un **sistema de control de versiones** es un software que registra los cambios realizados sobre un archivo o conjunto de archivos a lo largo del tiempo, de modo que sea posible recuperar cualquier versión anterior, comparar cambios, saber quién modificó qué y cuándo, y trabajar en paralelo sin sobrescribirse. Resuelve tres problemas reales de la industria: la **pérdida de trabajo** (todo cambio confirmado es recuperable), la **trazabilidad** (historial con autor, fecha y motivo) y la **colaboración** (varias personas sobre el mismo código sin pisarse).

Tipos de sistemas de control de versiones

Tipo	Cómo funciona	Ejemplos y limitación
Local	Base de datos de versiones en la propia máquina.	RCS. No permite colaborar.
Centralizado (CVCS)	Un único servidor guarda el historial; los clientes descargan solo la versión actual.	Subversion (SVN), CVS. Si el servidor cae o se corrompe, se pierde el historial; requiere conexión.
Distribuido (DVCS)	Cada persona clona el repositorio COMPLETO con todo su historial; se sincroniza con remotos.	Git, Mercurial. Es el estándar actual: trabajo sin conexión, cada clon es un respaldo completo, ramas baratas.

Contexto histórico: Git fue creado en 2005 por Linus Torvalds (creador de Linux) para gestionar el desarrollo del kernel de Linux. Hoy es el VCS dominante en la industria: prácticamente cualquier puesto de desarrollo web lo exige.

Git no es GitHub (y GitHub no es el único)

	Git	GitHub
¿Qué es?	La herramienta: software libre de control de versiones que corre en su computadora.	La plataforma: servicio en la nube que hospeda repositorios Git y agrega colaboración.
¿Qué aporta?	init, add, commit, branch, merge, historial...	Repositorios remotos, Pull Requests, Issues, revisión de código, perfil profesional (portafolio), GitHub Pages.
Alternativas	Mercurial (en desuso relativo)	GitLab, Bitbucket, Codeberg (mismo Git por debajo).

♦ En este curso, GitHub funcionará como **portafolio público de evidencias**: los reclutadores del sector revisan perfiles de GitHub. Un portafolio ordenado, con commits frecuentes y bien escritos, es un activo profesional real desde el primer día.

Los tres estados de Git: el concepto más importante de la sesión

Todo archivo dentro de un repositorio Git vive en un flujo de tres zonas. Dominar este modelo mental evita el 90 % de las confusiones de las personas principiantes:

Zona	Qué contiene	Comando que mueve hacia ella
1. Directorio de trabajo (working directory)	Los archivos tal y como los ve y edita en VS Code. Estado del archivo: modificado (modified) o sin seguimiento (untracked).	Editar / crear archivos
2. Área de preparación (staging area / index)	La “fotografía en borrador”: los cambios seleccionados que entrarán en la próxima confirmación. Estado: preparado (staged).	git add <archivo>
3. Repositorio (.git)	El historial permanente de confirmaciones. Estado: confirmado (committed).	git commit

Analogía recomendada: el directorio de trabajo es la **mesa donde se cocina**, el staging es el **plato que se emplata** (se decide qué va y qué no) y el commit es la **fotografía del plato terminado** que queda en el álbum para siempre. El staging existe para que cada commit sea **atómico**: un cambio lógico completo, no una mezcla de cosas sin relación.

Anatomía de un commit

Un **commit** es una instantánea (snapshot) completa del proyecto en un momento dado, identificada por un **hash SHA-1** de 40 caracteres (se usan los primeros 7 como abreviatura, p. ej. a3f9c2e). Cada commit guarda: la instantánea de los archivos, el autor y su correo (por eso configuramos la identidad), la fecha, el mensaje y un puntero a su commit padre — así se forma la cadena del historial. **HEAD** es el puntero que indica en qué commit (y rama) estamos parados.

Buenas prácticas profesionales de mensajes de commit (convención del curso)

El mensaje de commit es documentación para el futuro (y para la persona docente que evalúa el indicador “se evidencia el uso de versiones”). Adoptaremos desde hoy una versión simplificada de la convención de la industria **Conventional Commits** (conventionalcommits.org):

tipo: descripción breve en modo imperativo (máx. ~50 caracteres)

Tipos que usaremos en el curso:

feat: nueva funcionalidad o contenido	→ feat: agrega formulario de registro
fix: corrección de un error	→ fix: corrige enlace roto del menú
docs: documentación (README, bitácora)	→ docs: actualiza bitácora unidad 1
style: cambios de estilo/CSS o formato	→ style: aplica paleta de colores base
chore: mantenimiento (estructura, ignore)	→ chore: crea estructura de carpetas

- **Imperativo y descriptivo:** “agrega galería”, no “agregué cosas” ni “cambios”.
- **Atómico:** un commit = un cambio lógico. Si el mensaje necesita la palabra “y”, probablemente son dos commits.
- **Frecuente:** confirmar al terminar cada parte funcional, no una vez al final del día. En las evaluaciones, los commits periódicos son la evidencia de autoría y de manejo del tiempo.

Archivos de gobierno del repositorio: README.md y .gitignore

- **README.md:** la portada del repositorio; GitHub lo muestra automáticamente. Se escribe en Markdown (los archivos .md aparecerán en todo el curso). Debe decir qué es el proyecto, de quién es y cómo está organizado.
- **.gitignore:** lista de patrones de archivos que Git debe ignorar (temporales del sistema, carpetas de dependencias, comprimidos). Mantiene el historial limpio y profesional. Referencia de plantillas: github.com/github/gitignore.